Circular Lists

i) Insertion at start

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* insertAtStart(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (head == NULL) {
        newNode->next = newNode->prev = newNode;
        return newNode;
    }
    struct Node* last = head->prev;
    newNode->next = head;
    newNode->prev = last;
    last->next = head->prev = newNode;
    return newNode;
}

void display(struct Node* head) {
    if (!head) return;
    struct Node* temp = head;
    do {
        printf("%d <-> ", temp->data);
```

```
27          printf("%d <-> ", temp->data);
28          temp = temp->next;
29      } while (temp != head);
30      printf("(head)\n");
31  }
32
33  int main() {
34      struct Node* head = NULL;
35      head = insertAtStart(head, 20);
36      head = insertAtStart(head, 10);
37      display(head);
38      return 0;
39  }
```

Output

```
10 <-> 20 <-> (head)
```

ii) Insertion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* insertAtEnd(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (head == NULL) {
        newNode->next = newNode->prev = newNode;
        return newNode;
    }
    struct Node* last = head->prev;
    newNode->next = head;
    newNode->prev = last;
    last->next = head->prev = newNode;
    return head;
}

int main() {
    struct Node* head = NULL;
    head = insertAtEnd(head, 10);
    head = insertAtEnd(head, 20);
    return 0;
```

iii) Insertion at Position

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* insertAtPos(struct Node* head, int data, int pos) {
    if (pos == 1) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node
            ));
        newNode->data = data;
        if (!head) { newNode->next = newNode->prev = newNode; return
            newNode; }
        struct Node* last = head->prev;
        newNode->next = head; newNode->prev = last;
        last->next = head->prev = newNode;
        return newNode;
    }
    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp->next != head; i++) temp = temp
        ->next;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = temp->next;
    newNode->prev = temp;
```

```
        return newNode;
    }
    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp->next != head; i++) temp = temp
        ->next;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
    return head;
}

int main() {
    struct Node* head = NULL;
    head = insertAtPos(head, 10, 1);
    head = insertAtPos(head, 30, 2);
    head = insertAtPos(head, 20, 2);
    return 0;
}
```

Output

```
ircular List after position insertion: 10 -> 20 -> 30 -> (head)
```

iv) Deletion at start

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* deleteAtStart(struct Node* head) {
    if (head == NULL) return NULL;
    if (head->next == head) { free(head); return NULL; }
    struct Node* last = head->prev;
    struct Node* temp = head;
    head = head->next;
    head->prev = last;
    last->next = head;
    free(temp);
    return head;
}

int main() {
    struct Node* head = NULL; // Assume list has items
    head = deleteAtStart(head);
    return 0;
}
```

Output

Clear

```
Circular List after position insertion: 10 -> 20 -> 30 -> (head)


=== Code Execution Successful ===
```

v) Deletion at end

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* deleteAtEnd(struct Node* head) {
    if (head == NULL) return NULL;
    if (head->next == head) { free(head); return NULL; }
    struct Node* last = head->prev;
    last->prev->next = head;
    head->prev = last->prev;
    free(last);
    return head;
}

int main() {
    struct Node* head = NULL;
    head = deleteAtEnd(head);
    return 0;
}
```

Output

```
After deleting end, head is still: 10


=== Code Execution Successful ===
```

vi) Deletion at Position

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next, *prev;
};

struct Node* deleteAtPos(struct Node* head, int pos) {
    if (head == NULL) return NULL;
    if (pos == 1) {
        if (head->next == head) { free(head); return NULL; }
        struct Node* last = head->prev;
        struct Node* temp = head;
        head = head->next;
        head->prev = last; last->next = head;
        free(temp);
        return head;
    }
    struct Node* temp = head;
    for (int i = 1; i < pos && temp->next != head; i++) temp = temp
        ->next;
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
    return head;
}
```

```
22      temp->prev->next = temp->next;
23      temp->next->prev = temp->prev;
24      free(temp);
25      return head;
26  }
27
28  int main() {
29      struct Node* head = NULL;
30      head = deleteAtPos(head, 2);
31      return 0;
32  }
```

Output

```
After deleting pos 2, list continues from 10 to: 30


=== Code Execution Successful ===
```